

DATA STRUCTURES IMPLEMENTATION USING C++

Internship Assignment Report

Submitted By

Sanjay Pushparaj

Bachelor of Technology – Artificial Intelligence and Machine Learning

SNS College of Technology, Coimbatore

1. Introduction

Data Structures are fundamental concepts in computer science that enable efficient storage, organization, and manipulation of data. Every software application relies on appropriate data structures to achieve better performance and efficient memory utilization.

The objective of this assignment is to implement commonly used data structures using C++ and understand their internal working mechanisms. Instead of using built-in STL containers, each data structure has been implemented manually to gain a deeper understanding of insertion, deletion, searching, traversal, and data management operations.

This project includes the implementation of the following data structures:

1. Linked List
2. Stack
3. Queue
4. Binary Search Tree (BST)

The project was developed using C++ and compiled using the GNU G++ compiler.

2. Objectives

The primary objectives of this assignment are:

- To understand the importance of data structures in software development.
- To implement basic data structures from scratch.
- To perform common operations such as insertion, deletion, searching, and traversal.
- To analyze the time complexity of various operations.
- To gain hands-on experience with C++ programming.
- To learn compilation and execution using command-line tools.

3. Development Environment

- The following tools were used for development:
- ToolPurpose
- C++
- Programming Language
- G++ Compiler
- Compilation
- Visual Studio Code
- Code Editor
- Windows Command Prompt
- Program Execution
- GitHub
- Source Code Repository
- Operating System:
- Windows 11
- Compiler Version:
- g++ (MinGW-W64) 15.2.0

4. Project Structure

The project directory is organized as follows:

DataStructures/

```
|
|—— LinkedList/
|   |—— linkedlist.cpp
|
|—— Stack/
|   |—— stack.cpp
|
|—— Queue/
|   |—— queue.cpp
|
|—— BST/
|   |—— bst.cpp
|
|—— screenshots/
|
|—— README.md
```

This structure helps maintain separation between different implementations and improves readability.

5. Linked List Implementation

Description

A Linked List is a linear data structure consisting of nodes. Each node contains data and a pointer to the next node.

Unlike arrays, linked lists do not require contiguous memory allocation.

Operations Implemented

Insert

Adds a new node at the end of the list.

Delete

Removes a node containing a specified value.

Search

Checks whether a specific value exists in the list.

Display

Prints all elements in the list.

Time Complexity

Operation

Complexity

Insert

$O(n)$

Delete

$O(n)$

Search

$O(n)$

Display

$O(n)$

Sample Output

10 -> 20 -> 30 -> NULL

Search 20: Found

10 -> 30 -> NULL

Screenshot

Insert Linked List execution screenshot here.

6. Stack Implementation

Description

A Stack is a linear data structure that follows the Last In First Out (LIFO) principle.

The most recently inserted element is removed first.

Operations Implemented

Push

Adds an element to the top of the stack.

Pop

Removes the top element.

Peek

Displays the top element without removing it.

Display

Shows all elements currently present in the stack.

Time Complexity

Operation

Complexity

Push

$O(1)$

Pop

$O(1)$

Peek

$O(1)$

Sample Output

30 20 10

Top Element: 30

Popped: 30

20 10

Screenshot

Insert Stack execution screenshot here.

7. Queue Implementation

Description

A Queue is a linear data structure that follows the First In First Out (FIFO) principle.

The first inserted element is the first one to be removed.

Operations Implemented

Enqueue

Adds an element to the rear of the queue.

Dequeue

Removes an element from the front of the queue.

Display

Displays all elements in the queue.

Time Complexity

Operation

Complexity

Enqueue

$O(1)$

Dequeue

$O(1)$

Sample Output

10 20 30

Removed: 10

20 30

Screenshot

Insert Queue execution screenshot here.

8. Binary Search Tree (BST)

Description

A Binary Search Tree is a hierarchical data structure where:

- Left subtree contains smaller values.
- Right subtree contains larger values.
- Each node has at most two children.

BST provides efficient searching and insertion operations.

Operations Implemented

Insert

Adds a node according to BST properties.

Search

Locates a specific value in the tree.

Inorder Traversal

Displays values in sorted order.

Time Complexity

Operation

Average Case

Worst Case

Insert

$O(\log n)$

$O(n)$

Search

$O(\log n)$

$O(n)$

9. Compilation Commands

The programs were compiled using the GNU G++ compiler.

Linked List

```
g++ LinkedList\linkedlist.cpp -o LinkedList\linkedlist.exe
```

Stack

```
g++ Stack\stack.cpp -o Stack\stack.exe
```

Queue

```
g++ Queue\queue.cpp -o Queue\queue.exe
```

Binary Search Tree

```
g++ BST\bst.cpp -o BST\bst.exe
```

10. Execution Commands

Linked List

```
LinkedList\linkedlist.exe
```

Stack

```
Stack\stack.exe
```

Queue

```
Queue\queue.exe
```

Binary Search Tree

```
BST\bst.exe
```

11. Learning Outcomes

Through this assignment, the following concepts were learned:

- Dynamic memory allocation
- Pointer manipulation
- Linear data structures
- Non-linear data structures
- Tree traversal techniques
- Time complexity analysis
- Program compilation using G++
- Source code organization
- Practical implementation of fundamental data structures

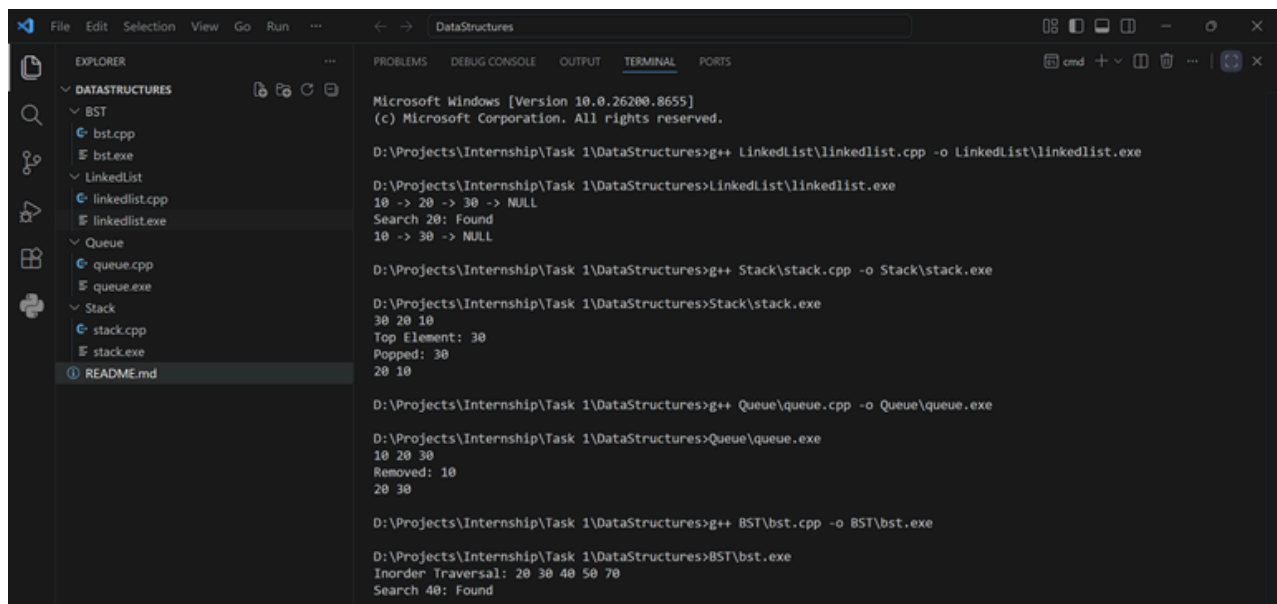
12. Conclusion

This project successfully implemented four fundamental data structures using C++. The implementation provided practical knowledge of how data is stored, managed, and accessed efficiently within a program.

Understanding these data structures is essential for software development, competitive programming, technical interviews, and advanced computer science concepts. The project strengthened programming skills, problem-solving abilities, and knowledge of algorithmic efficiency.

The assignment achieved all intended objectives and demonstrated the practical application of data structure concepts using C++.

Output



```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

D:\Projects\Internship\Task 1\DataStructures>g++ LinkedList\linkedList.cpp -o LinkedList\linkedList.exe

D:\Projects\Internship\Task 1\DataStructures>LinkedList\linkedList.exe
10 -> 20 -> 30 -> NULL
Search 20: Found
10 -> 30 -> NULL

D:\Projects\Internship\Task 1\DataStructures>g++ Stack\stack.cpp -o Stack\stack.exe

D:\Projects\Internship\Task 1\DataStructures>Stack\stack.exe
30 20 10
Top Element: 30
Popped: 30
20 10

D:\Projects\Internship\Task 1\DataStructures>g++ Queue\queue.cpp -o Queue\queue.exe

D:\Projects\Internship\Task 1\DataStructures>Queue\queue.exe
10 20 30
Removed: 10
20 30

D:\Projects\Internship\Task 1\DataStructures>g++ BST\bst.cpp -o BST\bst.exe

D:\Projects\Internship\Task 1\DataStructures>BST\bst.exe
Inorder Traversal: 20 30 40 50 70
Search 40: Found
```